PRD: Project: Softnix InsightOCR
Document Extraction Intelligence Platform

1. Project Overview

1.1 Product Vision

พัฒนา Web Application สำหรับจัดการและสกัดข้อมูลจากเอกสารอย่างชintelligent โดยใช้เทคโนโลยี OCR และ LLM เพื่อช่วยองค์กรในการประมวลผลเอกสาร ตรวจสอบความถูกต้อง และเชื่อมต่อกับระบบอื่นๆ ได้อย่างมีประสิทธิภาพ

1.2 Target Users

- **Admin**: ผู้ดูแลระบบองค์กร
- **Documents Admin**: ผู้จัดการ Schema และกำหนดเงื่อนไขการสกัดข้อมูล
- **Users**: ผู้ใช้งานทั่วไปที่ทำงานกับเอกสาร

1.3 Success Metrics

- ลดเวลาการประมวลผลเอกสารลง 70%
- ความแม่นยำในการสกัดข้อมูล > 95%
- User adoption rate > 80% ภายใน 3 เดือน
- ลด manual error จากการตรวจสอบเอกสาร > 90%

---

2. Technical Stack

2.1 Backend

- **Framework**: Python FastAPI
- **Database**: SQLite (with migration path to PostgreSQL)
- **OCR Engine**: Vision API กำหนดค่าใน .env
- **LLM Integration**: Self-hosted LLM รองรับ OpenAI API
- **File Storage**: Local filesystem / S3-compatible storage future

2.2 Frontend

- **Framework**: Next.js 14+ (App Router)
- **UI Components**: Cult UI + shadcn/ui v3
- **State Management**: React Context / Zustand

- **Form Validation**: React Hook Form + Zod
- **Styling**: Tailwind CSS

2.3 Infrastructure

- **Authentication**: JWT-based authentication
- **API Documentation**: OpenAPI (Swagger)
- **File Processing**: Background task queue (Celery / Redis)
- **Logging**: Structured logging with ELK stack (optional)

---

3. Functional Requirements

3.1 Authentication & Authorization

3.1.1 User Management

Features:

- ลงทะเบียนผู้ใช้ใหม่ (Admin เท่านั้น)
- เข้าสู่ระบบด้วย email/password
- รีเซ็ตรหัสผ่าน
- จัดการโปรไฟล์ส่วนตัว

User Roles:

Role	Permissions
Admin	- จัดการผู้ใช้ทั้งหมด - สร้าง/แก้ไข/ลบกลุ่ม/แผนก - เข้าถึงข้อมูลทั้งระบบ - ตั้งค่าระบบ
Documents Admin	- สร้าง/แก้ไข Document Schema - กำหนด Extraction Rules - ตั้งค่า Compliance Conditions - ดู Jobs ในแผนกของตน
User	- สร้าง/จัดการ Jobs - อัปโหลดเอกสาร - Review ผลการสกัด - Export ข้อมูล

3.1.2 Organization Structure

- สร้างกลุ่ม/แผนก (Departments)
- กำหนดผู้ใช้ในแต่ละกลุ่ม
- แยก Schema และ Jobs ตามกลุ่ม

3.2 Document Schema Management (Documents Admin)

3.2.1 Schema Creation

Features:

- สร้าง Schema ใหม่พร้อมชื่อและคำอธิบาย
- กำหนดประเภทเอกสาร (ใบเสนอราคา, สัญญา, ใบกำกับภาษี, ฯลฯ)
- เลือก OCR Engine ที่ต้องการใช้

3.2.2 Field Definition

Field Types:

- Text (ข้อความทั่วไป)
- Number (ตัวเลข)
- Date (วันที่)
- Currency (จำนวนเงิน)
- Boolean (ใช่/ไม่ใช่)
- Array (รายการ - เช่น รายการสินค้า)
- Object (กลุ่มข้อมูล)

Field Configuration:

```json
{
  "field_name": "total_amount",
  "field_label": "ยอดรวมทั้งสิ้น",
  "field_type": "currency",
  "required": true,
  "validation": {
    "min": 0,
    "max": 10000000
  },
  "extraction_prompt": "หายอดรวมทั้งสิ้นที่ระบุในเอกสาร รวม VAT"
```

}
```

#### 3.2.3 LLM Prompt Configuration
**Extraction Prompts:**
- กำหนด System Prompt สำหรับการสกัดข้อมูล
- กำหนด Field-specific Prompts
- รองรับ Few-shot examples
- ทดสอบ Prompt ด้วยเอกสารตัวอย่าง

**Example Prompt Template:**
```
คุณเป็นผู้เชี่ยวชาญในการสกัดข้อมูลจากเอกสาร
จากข้อความต่อไปนี้ ให้หา: {field_label}

เงื่อนไข:
- {extraction_rules}
- หากไม่พบข้อมูล ให้ใส่ null
- ตอบเป็น JSON format เท่านั้น

ข้อความจากเอกสาร:
{ocr_text}
```

### 3.2.4 Cross-Document Validation

### Features:

- อ้างอิงค่าจากเอกสารอื่นในชุดเดียวกัน
- ตรวจสอบความสอดคล้องระหว่างเอกสาร
- กำหนดเงื่อนไข Dependencies

### Example Rules:

```javascript
// ตรวจสอบว่า PO Number ในใบกำกับภาษีตรงกับ Purchase Order
{
  "rule_type": "cross_document",
```
```

```
  "source_document": "invoice",
  "source_field": "po_number",
  "target_document": "purchase_order",
  "target_field": "po_number",
  "condition": "equals",
  "error_message": "เลขที่ PO ไม่ตรงกัน"
}
```

3.2.5 Compliance & Baseline Validation

Compliance Rules:

- ตรวจสอบตามมาตรฐานที่กำหนด (เช่น ภาษีต้องเป็น 7%)
- เปรียบเทียบกับ Baseline values
- Alert เมื่อค่าผิดปกติ

Example Compliance:

javascript

```
{
  "rule_name": "vat_check",
  "description": "ตรวจสอบอัตราภาษี VAT",
  "field": "vat_rate",
  "operator": "equals",
  "expected_value": 0.07,
  "severity": "error"
}
```

3.3 User Workflow

Step 1: Create Jobs

Features:
- สร้าง Job ใหม่พร้อมชื่อและคำอธิบาย
- เลือก Document Schema ที่ต้องการใช้

- กำหนดจำนวนเอกสารที่คาดหวัง (optional)
- เลือกแผนก/กลุ่มที่รับผิดชอบ

UI Elements:
```
หน้า: Jobs List
- ปุ่ม "+ สร้าง Job ใหม่"
- ตารางแสดง Jobs ทั้งหมด (ชื่อ, Schema, สถานะ, วันที่สร้าง)
- Filter: ตาม Schema, สถานะ, วันที่
- Search: ค้นหาชื่อ Job

Modal: สร้าง Job
- ชื่อ Job (required)
- เลือก Schema (dropdown)
- คำอธิบาย (textarea)
- จำนวนเอกสารโดยประมาณ (number)
- ปุ่ม "สร้าง"
```

Step 2: Upload Documents

Features:
- อัปโหลดเอกสารหลายไฟล์พร้อมกัน (drag & drop / browse)
- รองรับไฟล์: PDF, JPG, PNG, TIFF
- แสดงรายการเอกสารที่อัปโหลดแล้ว
- ระบุสถานะแต่ละไฟล์ (รออัปโหลด, กำลังประมวลผล, พร้อมสกัด, เสร็จสิ้น)
- อัปโหลดเพิ่มเติมได้ตลอด

File Processing:
1. อัปโหลดไฟล์ → เก็บใน storage
2. ทำ OCR อัตโนมัติ
3. เปลี่ยนสถานะเป็น "พร้อมสกัด"

UI Elements:

```
หน้า: Job Detail - Upload Tab
- Drag & drop zone
- รายการเอกสาร (thumbnail, ชื่อไฟล์, ขนาด, สถานะ)
- Progress bar สำหรับการอัปโหลดและ OCR
- ปุ่ม "อัปโหลดเพิ่ม"
```

Step 3: Extract Data

Features:

- แสดงเอกสารที่พร้อมสกัดทั้งหมด
- เลือกสกัดทีละไฟล์หรือทั้งหมดพร้อมกัน
- แสดง Progress real-time
- Retry กรณี error

Extraction Process:

1. ดึง OCR text จาก database
2. ส่งไปยัง LLM พร้อม Schema prompts
3. Parse JSON response
4. บันทึกผลลัพธ์ใน database
5. เปลี่ยนสถานะเป็น "รอตรวจสอบ"

UI Elements:

```
หน้า: Job Detail - Extract Tab
- รายการเอกสาร + checkbox
- ปุ่ม "สกัดที่เลือก" / "สกัดทั้งหมด"
- Progress indicator
- Error log (หากมี)
```

Step 4: Review & Validate

Features:

- แสดงเอกสารทีละไฟล์
- แสดง Original document (PDF viewer) และผลการสกัด (side by side)
- แก้ไขข้อมูลที่สกัดได้
- ไฮไลท์ field ที่มี confidence ต่ำ
- บันทึกการแก้ไข

Data Display:

```
Layout: 2 columns
- ซ้าย: PDF Viewer แสดงเอกสารต้นฉบับ
- ขวา: Form แสดงข้อมูลที่สกัดได้
  - แต่ละ field มี confidence score
  - แก้ไขได้โดยตรง
  - ปุ่ม "บันทึก" / "ถัดไป"
```

Validation Indicators:

- 🟢 สีเขียว: Confidence > 90%
- 🟡 สีเหลือง: Confidence 70-90%
- 🔴 สีแดง: Confidence < 70% หรือ validation error

Step 5: Condition Checking

Features:

- รัน Compliance rules อัตโนมัติ
- รัน Cross-document validation
- แสดงผลการตรวจสอบแต่ละ rule

- รายงานความสอดคล้อง

Condition Types:

1. **Field Validation**: ตรวจสอบค่าแต่ละ field (data type, range, format)
2. **Cross-field Validation**: ตรวจสอบความสัมพันธ์ระหว่าง field (เช่น amount = quantity × unit_price)
3. **Cross-document Validation**: เปรียบเทียบข้อมูลระหว่างเอกสาร
4. **Compliance Rules**: ตรวจสอบตามเงื่อนไขที่กำหนดใน Schema

UI Elements:

```
หน้า: Job Detail - Conditions Tab
- สรุปผลการตรวจสอบ (Passed / Failed / Warning)
- รายละเอียดแต่ละ rule:
  - ชื่อ rule
  - สถานะ (✓ ผ่าน / ✗ ไม่ผ่าน / ⚠️ แจ้งเตือน)
  - เอกสารที่เกี่ยวข้อง
  - ข้อความอธิบาย
- Filter: แสดงเฉพาะที่ไม่ผ่าน / ทั้งหมด
```

Example Report:

```
รายงานการตรวจสอบ Job: "ใบเสนอราคา Q1/2025"

✓ ผ่าน (12 rules)
✗ ไม่ผ่าน (2 rules)
  - PO Number ไม่ตรงกัน: Invoice #001 vs PO #002
  - ยอดรวมเกินงบประมาณที่อนุมัติ: 1,500,000 บาท > 1,000,000 บาท

⚠️ แจ้งเตือน (1 rule)
  - วันที่ครบกำหนดใกล้ถึง: เหลือ 3 วัน
```

Step 6: Actions

Features:
- Export ข้อมูลในรูปแบบต่างๆ
- ส่งข้อมูลผ่าน Webhook / API
- Download เอกสารต้นฉบับพร้อมข้อมูลที่สกัด

Export Options:
1. **Excel (.xlsx)**: ทุก field ในแถวเดียว
2. **JSON**: Structured data
3. **CSV**: Flat format
4. **PDF Report**: Summary report พร้อมภาพเอกสาร

Integration Actions:
- เลือก Integration ที่ตั้งค่าไว้
- ตรวจสอบ payload ก่อนส่ง
- ส่งข้อมูล
- แสดงผลการส่ง (สำเร็จ/ล้มเหลว)

UI Elements:
```
หน้า: Job Detail - Actions Tab

ส่วนที่ 1: Export
- ปุ่ม "ดาวน์โหลด Excel"
- ปุ่ม "ดาวน์โหลด JSON"
- ปุ่ม "ดาวน์โหลด CSV"
- ปุ่ม "สร้างรายงาน PDF"

ส่วนที่ 2: Integration
- Dropdown เลือก Integration
- ปุ่ม "ดูตัวอย่าง Payload"
- ปุ่ม "ส่งข้อมูล"
```

- Log แสดงประวัติการส่ง

3.4 Integration Management

3.4.1 Webhook Configuration

Features:

- ตั้งค่า Webhook URL
- กำหนด HTTP Method (POST/PUT)
- ตั้งค่า Headers (Authorization, Content-Type)
- กำหนด Payload template
- ทดสอบการเชื่อมต่อ

Configuration Example:

```json
{
  "integration_name": "ERP System",
  "type": "webhook",
  "url": "https://erp.example.com/api/invoices",
  "method": "POST",
  "headers": {
    "Authorization": "Bearer ${API_KEY}",
    "Content-Type": "application/json"
  },
  "payload_template": {
    "invoice_number": "{{invoice.invoice_number}}",
    "total": "{{invoice.total_amount}}",
    "items": "{{invoice.line_items}}"
  },
  "retry_config": {
    "max_retries": 3,
    "retry_delay": 5
  }
}
```

3.4.2 Integration Types

- **Webhook**: ส่ง HTTP request ไปยัง external API
- **API Integration**: เชื่อมต่อกับ API ที่กำหนด (future)
- **Database**: เขียนข้อมูลลง external database (future)
- **File Export**: ส่งไฟล์ไปยัง FTP/SFTP (future)

---

4. Non-Functional Requirements

4.1 Performance

- การสกัดข้อมูล 1 หน้า < 5 วินาที
- รองรับ concurrent users 50+ คน
- อัปโหลดไฟล์ได้สูงสุด 50 MB/file
- ประมวลผลไฟล์ได้พร้อมกันสูงสุด 10 ไฟล์

4.2 Security

- Authentication: JWT with refresh token
- Password: bcrypt hashing, minimum 8 characters
- File upload: Validate file type และ scan malware
- API: Rate limiting (100 requests/minute/user)
- Data: Encrypt sensitive fields in database

4.3 Usability (UX Writing)

3C Framework:

- **Clear (ชัดเจน)**: ใช้ภาษาที่เข้าใจง่าย หลีกเลี่ยงศัพท์เทคนิค
- **Concise (กระชับ)**: ใช้คำน้อยที่สุดที่สื่อความหมายครบ
- **Consistent (สม่ำเสมอ)**: ใช้คำเดิมสำหรับสิ่งเดียวกันทั้งระบบ

ตัวอย่าง:

- ❌ "กรุณา upload document เพื่อดำเนินการ extract"
- ✅ "อัปโหลดเอกสาร"
- ❌ "Process completed successfully"
- ✅ "สกัดข้อมูลเสร็จสิ้น"

4.4 Accessibility

- Keyboard navigation support

- Screen reader friendly
- High contrast mode
- Responsive design (desktop-first)

---

5. UI/UX Design Guidelines

5.1 Design System

Framework: Cult UI + shadcn/ui v3 **Reference**: https://www.cult-ui.com/docs/installation

5.2 Color Palette

Primary Colors:

- Black: `#000000` (text, borders, primary actions)
- White: `#FFFFFF` (backgrounds, cards)
- Gray scale: `#F5F5F5`, `#E5E5E5`, `#A3A3A3`, `#525252`

Functional Colors (ใช้เฉพาะจุดสำคัญ):

- Success: `#10B981` (Green) - การดำเนินการสำเร็จ
- Warning: `#F59E0B` (Amber) - คำเตือน, ต้องตรวจสอบ
- Error: `#EF4444` (Red) - ข้อผิดพลาด
- Info: `#3B82F6` (Blue) - ข้อมูล, focus state

5.3 Typography

- **Font**: Inter (default) หรือ Noto Sans Thai
- **Headings**:
  - H1: 32px, Bold
  - H2: 24px, Semibold
  - H3: 20px, Semibold
- **Body**: 16px, Regular
- **Small**: 14px, Regular
- **Caption**: 12px, Regular

5.4 Layout Principles

- **Minimal**: ลดองค์ประกอบที่ไม่จำเป็น
- **White space**: ใช้ white space อย่างเหมาะสม
- **Grid**: 12-column grid system
- **Spacing**: 4px increment (4, 8, 12, 16, 24, 32, 48, 64)

5.5 Component Guidelines

Buttons:

tsx

```tsx
// Primary action
<Button variant="default">สกัดข้อมูล</Button>

// Secondary action
<Button variant="outline">ยกเลิก</Button>

// Destructive action
<Button variant="destructive">ลบ</Button>
```

Forms:

- ใช้ label ชัดเจน
- แสดง error message ใกล้ field ที่มีปัญหา
- ใช้ placeholder เฉพาะตัวอย่าง
- Required fields: ใช้ * หรือข้อความ "(ต้องระบุ)"

Tables:

- Zebra striping (optional)
- Hover state
- Sortable columns
- Pagination

Cards:

- Border: 1px solid gray
- Padding: 16-24px
- Shadow: subtle (optional)

---

6. Data Models

6.1 Core Entities

User

python

```
{
  "id": "uuid",
  "email": "string",
  "password_hash": "string",
  "full_name": "string",
  "role": "admin | documents_admin | user",
  "department_id": "uuid",
  "is_active": "boolean",
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

Department

python

```
{
  "id": "uuid",
  "name": "string",
  "description": "string",
  "created_at": "timestamp"
}
```

DocumentSchema

python

```
{
  "id": "uuid",
  "name": "string",
  "description": "string",
  "document_type": "string",
  "ocr_engine": "tesseract | vision_api | textract",
  "fields": [
    {
      "field_name": "string",
      "field_label": "string",
      "field_type": "text | number | date | currency | boolean | array | object",
      "required": "boolean",
```

```
    "validation": {},
    "extraction_prompt": "string"
  }
 ],
 "compliance_rules": [],
 "cross_document_rules": [],
 "created_by": "uuid",
 "department_id": "uuid",
 "created_at": "timestamp",
 "updated_at": "timestamp"
}
```

Job

python

```
{
 "id": "uuid",
 "name": "string",
 "description": "string",
 "schema_id": "uuid",
 "status": "draft | processing | review | completed | failed",
 "created_by": "uuid",
 "department_id": "uuid",
 "created_at": "timestamp",
 "updated_at": "timestamp"
}
```

Document

python

```
{
 "id": "uuid",
 "job_id": "uuid",
 "filename": "string",
 "file_path": "string",
 "file_size": "integer",
 "mime_type": "string",
```

```
  "status": "uploaded | ocr_processing | ocr_completed | extraction_pending | extraction_completed | reviewed | failed",
  "ocr_text": "text",
  "ocr_confidence": "float",
  "extracted_data": "json",
  "reviewed_data": "json",
  "extraction_confidence": "float",
  "uploaded_at": "timestamp",
  "processed_at": "timestamp"
}
```

ValidationResult

python

```
{
  "id": "uuid",
  "job_id": "uuid",
  "rule_type": "field | cross_field | cross_document | compliance",
  "rule_name": "string",
  "status": "passed | failed | warning",
  "severity": "error | warning | info",
  "message": "string",
  "related_documents": ["uuid"],
  "created_at": "timestamp"
}
```

Integration

python

```
{
  "id": "uuid",
  "name": "string",
  "type": "webhook | api | database | file",
  "config": {
    "url": "string",
    "method": "string",
    "headers": {},
    "payload_template": {}
```

```
  },
  "department_id": "uuid",
  "created_by": "uuid",
  "created_at": "timestamp"
}
```

IntegrationLog

python

```
{
  "id": "uuid",
  "integration_id": "uuid",
  "job_id": "uuid",
  "status": "success | failed",
  "request_payload": "json",
  "response": "json",
  "error_message": "string",
  "created_at": "timestamp"
}
```

---

7. API Endpoints

7.1 Authentication

```
POST   /api/auth/login
POST   /api/auth/logout
POST   /api/auth/refresh
POST   /api/auth/reset-password
```

7.2 Users

```
GET    /api/users
POST   /api/users
```

```
GET    /api/users/{id}
PUT    /api/users/{id}
DELETE /api/users/{id}
```

7.3 Departments

```
GET    /api/departments
POST   /api/departments
GET    /api/departments/{id}
PUT    /api/departments/{id}
DELETE /api/departments/{id}
```

7.4 Document Schemas

```
GET    /api/schemas
POST   /api/schemas
GET    /api/schemas/{id}
PUT    /api/schemas/{id}
DELETE /api/schemas/{id}
POST   /api/schemas/{id}/test-prompt
```

7.5 Jobs

```
GET    /api/jobs
POST   /api/jobs
GET    /api/jobs/{id}
PUT    /api/jobs/{id}
DELETE /api/jobs/{id}
GET    /api/jobs/{id}/documents
POST   /api/jobs/{id}/upload
POST   /api/jobs/{id}/extract
GET    /api/jobs/{id}/validation-results
POST   /api/jobs/{id}/export
```

```
POST   /api/jobs/{id}/send-integration
```

7.6 Documents

```
GET    /api/documents/{id}
PUT    /api/documents/{id}
DELETE /api/documents/{id}
POST   /api/documents/{id}/extract
PUT    /api/documents/{id}/review
```

7.7 Integrations

```
GET    /api/integrations
POST   /api/integrations
GET    /api/integrations/{id}
PUT    /api/integrations/{id}
DELETE /api/integrations/{id}
POST   /api/integrations/{id}/test
GET    /api/integrations/{id}/logs
```

---

8. User Journeys

8.1 Documents Admin: สร้าง Schema

1. เข้าสู่ระบบด้วย Documents Admin role
2. ไปที่ "จัดการ Schema"
3. คลิก "+ สร้าง Schema ใหม่"
4. กรอกชื่อ Schema และเลือกประเภทเอกสาร
5. เพิ่ม Fields:
    - กรอกชื่อ field
    - เลือกประเภทข้อมูล
    - เขียน extraction prompt
    - กำหนด validation rules
6. ตั้งค่า Compliance rules
7. ตั้งค่า Cross-document validation (ถ้ามี)

8. ทดสอบ Schema ด้วยเอกสารตัวอย่าง
9. บันทึก Schema

8.2 User: ประมวลผลเอกสาร

1. เข้าสู่ระบบด้วย User role
2. ไปที่ "งาน" (Jobs)
3. คลิก "+ สร้าง Job ใหม่"
4. กรอกรายละเอียด
5. คลิก "สร้าง"
6. อัปโหลดเอกสาร (drag & drop หรือ browse)
7. เลือกทีละเอกสารเพื่อสกัด หรือเลือกทั้งหมด รอ OCR เสร็จ
8. เลือก schema ได้ในแต่ละไฟล์ กดทดสอบ โดยตรวจสอบกับ validation rules
9. เลือก Review เอกสารทีละไฟล์ หรือคลิกตรวจสอบการ validation rules
10. แก้ไขข้อมูลที่ผิดพลาด (ถ้ามี)
11. บันทึกการแก้ไข
12. คลิกตรวจสอบเงื่อนไข Compliance หรือ Cross-document validation
13. ผลลัพธ์ % ความสอดคล้อง
14. Export หรือส่งข้อมูลผ่าน Integration
15. ดาวน์โหลดผลลัพธ์

8.3 Admin: จัดการผู้ใช้

1. เข้าสู่ระบบด้วย Admin role
2. ไปที่ "ผู้ใช้"
3. คลิก "+ เพิ่มผู้ใช้"
4. กรอกข้อมูลผู้ใช้
5. เลือก Role
6. เลือกแผนก
7. บันทึก
8. ส่ง email แจ้งผู้ใช้ใหม่

---

11. Deployment & Infrastructure

11.1 Development Environment

- Local development with Docker Compose
- Hot reload for backend and frontend

- SQLite database

11.2 Staging Environment

- Docker containers
- PostgreSQL database
- S3-compatible storage
- Redis for task queue

11.3 Production Environment

- Kubernetes cluster (optional) or Docker Swarm
- PostgreSQL with replication
- S3 for file storage
- Redis for caching and task queue
- Load balancer
- SSL/TLS certificates

11.4 Monitoring & Logging

- Application logs: Structured JSON logging
- Error tracking: Sentry (optional)
- Performance monitoring: Prometheus + Grafana (optional)
- Uptime monitoring: UptimeRobot (optional)

---

12. Security Considerations

12.1 Data Protection

- Encrypt sensitive data at rest
- Use HTTPS for all communications
- Sanitize user inputs
- Validate file uploads (type, size, content)

12.2 Access Control

- Role-based access control (RBAC)
- Department-level data isolation
- Audit logging for sensitive operations

12.3 API Security

- JWT authentication

- Rate limiting
- CORS configuration
- Input validation

12.4 File Security

- Malware scanning
- File type validation
- Size limits
- Secure file storage with signed URLs

---

13. Future Enhancements

13.1 Short-term (3-6 months)

- Advanced OCR: Handwriting recognition
- More LLM providers: Google, AWS Bedrock
- Batch job scheduling
- Email notifications
- Audit trail dashboard

13.2 Medium-term (6-12 months)

- Mobile application
- Advanced analytics and reporting
- Custom LLM fine-tuning
- Multi-language support
- Advanced workflow automation

13.3 Long-term (12+ months)

- AI-powered document classification
- Automatic schema generation
- Real-time collaboration
- Advanced data visualization
- Integration marketplace

---

14. Success Criteria

14.1 Technical

- ✅ System uptime > 99.5%
- ✅ API response time < 200ms (p95)
- ✅ Extraction accuracy > 95%
- ✅ Zero security vulnerabilities

14.2 Business

- ✅ 80% user adoption within 3 months
- ✅ 70% reduction in manual processing time
- ✅ 90% reduction in manual errors
- ✅ Positive user feedback (NPS > 50)

14.3 User Experience

- ✅ Task completion rate > 90%
- ✅ User satisfaction score > 4/5
- ✅ Low support ticket volume (< 5/week)

---

15. Risks & Mitigation

15.1 Technical Risks

Risk	Impact	Mitigation
LLM API instability	High	Implement retry logic, fallback providers
OCR accuracy issues	Medium	Support multiple OCR engines, manual review
Large file processing	Medium	Implement chunking, async processing
Database performance	Medium	Add indexes, implement caching

15.2 Business Risks

Risk	Impact	Mitigation
Low user adoption	High	User training, simple UX, support documentation
Data privacy concerns	High	Clear privacy policy, data encryption, compliance
Integration complexity	Medium	Standard API, comprehensive documentation

---

16. Appendix

16.1 Glossary

- **OCR**: Optical Character Recognition - เทคโนโลยีแปลงภาพเป็นข้อความ

- **LLM**: Large Language Model - โมเดลภาษาขนาดใหญ่
- **Schema**: โครงสร้างข้อมูลที่กำหนดไว้
- **Extraction**: การสกัดข้อมูลจากเอกสาร
- **Compliance**: การตรวจสอบความสอดคล้องตามกฎเกณฑ์
- **Webhook**: การส่งข้อมูลไปยัง URL อื่นผ่าน HTTP

16.2 References

- Cult UI Documentation: https://www.cult-ui.com/docs
- FastAPI Documentation: https://fastapi.tiangolo.com
- Next.js Documentation: https://nextjs.org/docs
- shadcn/ui: https://ui.shadcn.com

Document Version: 1.0
Last Updated: 2025-01-29
Prepared By: Rujirapong
Status: Draft for Review